Sorting Algorithms

1

Topics

Introduction

Insertion Sort Algorithm

Analysis of Insertion Sort

Quick Sort

Randomized Quick Sort

Analysis of Quick Sort

Merge Sort

Analysis of Merge Sort

2

Sorting Algorithms

Classification

The sorting algorithms are classified into two categories on the basis of ***underlying procedure*** used:

Sorting by comparison: These are based on comparison of keys . The method has general applicability. Examples are selection sort, quick sort, etc.

Sorting by counting: These depend on the characteristics of individual data items. The sort method has limited application. Examples include radix sort, bucket sort, sort by counting

The algorithms are also categorized on the basis of ***storage type*** used to implement the procedure

Internal Sorting : RAM is used for storage and processing. Execution speed is fast. Used by operating systems, Real time systems

External Sorting: RAM and Secondary Storage are used. Comparatively slow execution. Used by DBMS for large databases, Word Processors

3

Sorting Algorithms

Sorting by Comparison

The ***Sorting by Comparison*** method sorts input by comparing pairs of keys in the input. Generally, the comparison ***sorting is not stable***. (An algorithm is called ***stable*** if it preserves the order of sub-keys for any two equal keys. A stable algorithm, for example, sorts the salaries of employees such that employees with equal salary appear in alphabetic order).
The comparison sorting are categorized as ***elementary*** and ***advanced.***

Elementary Algorithms: The elementary algorithm are ***inefficient*** but ***easy to implement.***
Their running times are $\theta(n^2)$.Common algorithms are
Exchange sort
Selection sort
Insertion sort
Shell sort

Advanced Algorithms: Advanced algorithms are *efficient*. The implementations are usually based on ***recursive function calls***. Their running times are $\theta(nlg\ n)$. The typical advanced algorithms include
Merge sort
Quick sort
Heap sort

4

- 10 5 11 5 6 9 5 3
- 3 5 5 5 6 9 10 11

5

Sorting Algorithms

Sorting by Counting

Sorting by counting algorithms are used to arrange data items on the basis of ***distribution of digits/characters*** in individual data items. The common methods are

Simple Counting

Radix sort

Bucket sort

The sorting techniques largely depend on the composition and range of data items being sorted

Sorting by counting provides running time $\theta(n)$, for certain categories of data sets

Sorting by counting is ***stable***.

6

Insertion Sort

7

Insertion Sort

Algorithm

The algorithm for ***insertion sorting*** of an array ***A(1..n]*** is as follows:

*The element **A[2] is selected as key**.*
The key is compared with A[1]. If key is less, the element A[1] is shifted towards right, and key is inserted in the vacant cell

*The element **A[3] is selected as key.** The **sub-array A[2], A[1] is scanned** to locate an element smaller than key. The elements larger than key are shifted to right.*
*The key is **inserted after an element that is smaller than key**.*

*The element **A[4] is selected as key**. The **sub-array A[3],[2], A[1] is scanned** to locate an element smaller than key. The elements larger than key are shifted towards right.*
*The key is **inserted after an element that is smaller than key**.*

- -

*The element **A[n] is selected as key.** The sub-array **A[n-1]… A[2], A[1]** is scanned to locate an element smaller than key. The elements larger than key are shifted towards right.*
*The key is **inserted after an element that is smaller than key**.*

The array ***A[1..n]*** is now sorted in ***ascending order***

8

Insertion Sort

Pseudo Code

The **INSERT-SORT** method sorts an input array *A[1…n]* using ***insertion sort*** method.

INSERT-SORT(A)

```
1  n ← length[A]                    ► n holds array size
2  for j ← 2 to n  do               ► Examine elements A[2] ….A[n]
3      key ← A[j]                   ► Select key
4       i ← j - 1
5      while i > 0 and A[i] > key  do   ► Scan backwards  for an element smaller than key
6          A[i+1]  ← A[i]           ► Shift toward right to create space for insertion of key
7          i ← i-1
8      A[i+1] ← key                 ► insert key in the vacant place
```

9

Analysis of Insertion Sort

10

Insertion Sort Analysis

Best Case Running Time

The ***best case*** occurs when the input array is ***presorted***

A[1]	A[2]	A[3]	A[4]	A[5]	-	-	A[n-1]	A[n]
5	15	37	45	58	-	-	87	92

Comparison of ***key***

If A[2] is selected as key, only ***one*** *comparison is done*

If A[3] is selected as key ,only ***one*** *comparison is done*

If A[4] is selected as key ,only ***one*** *comparison is done*

- -

If A[n] is selected as key ,only ***one*** *comparison is done*

Thus, altogether ***n-1 comparisons*** are done

The ***best case running time*** of insertion sort is $\theta(n-1)=\theta(n)$

11

Insertion Sort Analysis

Worst Case Running Time

The ***worst case*** occurs when the input array is ***reverse sorted*** .

A[1]	A[2]	A[3]	A[4]	A[5]	-	-	A[n-1]	A[n]
95	85	66	55	43	-	-	10	8

If A[2] is selected as key, only ***one*** comparison/shift is done

If A[3] is selected as key ,only **two** comparisons/shifts are done

If A[4] is selected as key ,only ***three*** comparisons/shifts are done

- -

If A[n] is selected as key , ***n-1*** comparisons/shifts are done

Thus, if *c* is unit cost, the total cost, $T_w(n)$, for the whole array is given by
$T_w(n)=c[1 + 2 + 3+ 4+\ldots\ldots\ldots\ldots(n-2) +(n-1)]=c.\ n(n-1)/2$

The ***worst case*** running time $T_w(n)$ of insertion sort is $\theta(\ n(n-1)/2\ )=\theta(n^2)$

12

Insertion Sort Analysis

Average Case Running Time-I

In order to find the ***average running time***, we first consider the ***expected cost*** of inserting an element in a ***subarray of k elements,*** by using probabilistic analysis.

A[1]	A[2	A[3]	A[4]	A[5]	-	A[k-1]	A[k]	A[k+1]	A[k2]			A[n]

The insertion sort procedure looks for the position of an element ***just smaller than the key*** . Assuming that the requisite element has equal probability of being in any of the *k* locations, the ***probability of finding the desired element in any location, in the subarray, is 1/k.***

Each ***insertion of key causes one shift*** to the right . Assuming that c is unit cost, the table summarizes the number of shift operations and associated costs.

Location of element smaller than *key*	probability	# of shifts operations	Cost
k	*1/k*	*1*	*c*
k-1	*1/k*	*2*	*2c*
k-2	*1/k*	*3*	*3c*
...	...	...	
3	*1/k*	*k-2*	*(k-2)c*
2	*1/k*	*k-1*	*(k-1)c*
1	*1/k*	*k*	*kc*

Let $T_e(k)$ be the ***expected cost of inserting key into subrray of k elements***. Then,

$$T_e(k) = (1/k)c + (1/k).2c + \ldots\ldots + (1/k).(k-1)c + (1/k).kc$$
$$= (c/k)[1+2+3+\ldots\ldots\ldots+(k-1)+k] = (c/k)[k(k+1)/2]$$
$$= c(k+1)/2$$

13

Insertion Sort Analysis

Average Case Running Time-II

The ***total cost of insertions into the complete array of size n*** is determined by summing expected costs for subarrays. If $T_a(n)$ is the average running time for the insertion sort then

$$T_a(n) = c.\left[\sum_{k=2}^{n}(k+1)/2\right] = \left[\sum_{k=2}^{n} k/2 + \sum_{k=2}^{n} 1/2\right]$$

By evaluating the summations,

$$T_a(n) = c.(n^2 + 3n - 4)/4 = \theta(n^2)$$

The ***expected running time*** of insertion sort is $\theta(n^2)$

14

Quick Sort

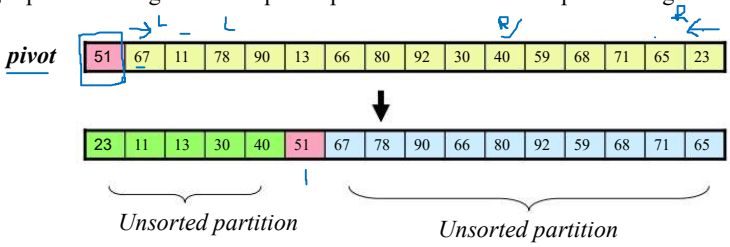
15

Quick Sort

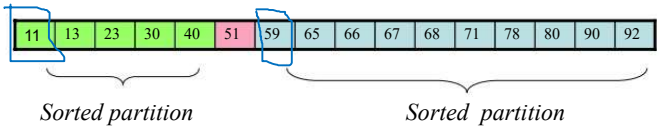
Procedure

The quick sort procedure works in two phases, called ***partitioning*** and ***recursive sorting***

Phase #1: In Phase #1, an array element is chosen as a ***pivot*** The array is partitioned into two subarrays such that all elements in left partition are ***less than pivot***, and all elements in right partition are greater or equal to pivot. An illustration of partitioning is as follows:



Phase #2: In Phase #2, the left and right subarrays are sorted recursively



16

Quick Sort

Partitioning Algorithm

A common ***partitioning procedure***, due originally to Hoare , is as follows

Step #1: *Select left most element as* ***pivot***

Step #2: *Use* ***i*** *as* ***left pointer,*** *and* ***j*** *as* ***right pointer*** *to scan the array*

Step #3; *Scan array from* ***right-to-left, by*** *decreasing j, until an element smaller than pivot is encountered*

Step#4: *Scan array from* ***left-to-right,*** *by increasing* ***i,*** *until an element larger than pivot is encountered*

Step#5: *Exchange the elements pointed by* ***i*** *and* ***j.***

Step #6: *Move pointers forward (increase* ***i*** *by 1, and decrease* ***j*** *by 1)*

Step #7 *Repeat Step#3 through Step#6 until pointers cross-over*

Step#8: *Exchange pivot with element identified by the left pointer*

17

Quick Sort

Pseudo Code-I

Quick sort is implemented using two procedures: **QUICKSORT** and **PARTITION**

```
QUICKSORT(A, p ,r)
1   if  p < r                          ► Recursion is terminated when boundaries cross-over
2     then q ← PARTITION(A, p,r )      ► The PARTITION method returns index of pivot
3        QUICKSORT(A, p, q-1)          ►sort left partition recursively
4        QUICKSORT(A, q+1, r)          ►sort right partition recursively
```

18

Quick Sort

Pseudo Code-II

The **PARTITION** uses two counters to scan the array from ***left-to-right*** and then from ***right-to-left*** The first element is chosen as the pivot

```
PARTITION( A, p, r)
1  x ← A[p]          ► left-most element is chosen as pivot. It is stored in x
2  i ← p-1           ► i scans the array from left-to-right
3  j ←r+1            ►j  scans the array from right-to-left
4  while TRUE  do    ►An infinite loop is set up
5     repeat j ← j-1
6     until A[j] ≤ x
7     repeat i ←i+1
8     until A[i] ≥ x
9     if i < j                    ► loop terminates when pointers  cross-over
10       then exchange A[i]↔A[j]  ► Exchange elements identified by i and j
11    else return j               ► Return index of pivot
```

19

Quick Sort

Pivot Selection

Choice of pivot ***influences the performance*** of quick sort algorithm Some of the choices are:

Left-most element: The first element is chosen as pivot. There is chance of bad partitioning

1	2	3	4	5	6	7	8	9	10
56	90	82	88	12	50	89	22	77	25

pivot=56

Right-most element: The last element is chosen as pivot There is chance of bad partitioning.

1	2	3	4	5	6	7	8	9	10
56	90	82	88	12	50	89	22	77	25

pivot=25

Median element: An element with index that is median of left-most and right-most element index. If *l* and *r* are the indexes of first and last element, median index *m* is computed as

$m = \lfloor (l + r) / 2 \rfloor = \lfloor (1+10)/2 \rfloor = 5$

1	2	3	4	5	6	7	8	9	10
56	90	82	88	12	50	89	22	77	25

pivot=12

The choice of median minimizes the chances of bad partitioning.

Random element: An element is selected randomly. A random number generator is used to compute the index of element being chosen .For example, if the function *random (l,r)* returns a random number In the range *l* to *r*, the index *i* of the selected element is given by

i = random(l ,r) = (1,10)= 7, say

1	2	3	4	5	6	7	8	9	10
56	90	82	88	12	50	89	22	77	25

pivot=89

The random selection ***minimizes*** the chances of bad partitioning

20

Quick Sort

Randomized Algorithm

In ***randomized quick sort***, the pivot is chosen ***randomly*** from the elements in subarray. A ***random number generator*** is used to generate an index which lies in the range of upper and lower indexes of the array. The procedures for partitioning and recursive sorting are given below.

```
RANDOMIZED-PARTITION( A, p, r)
1  i← RANDOM(p,r)          ►RANDOM generates a random number in range  p to r
2  exchange A[l]↔A[i]      ►Exchange left-most element with the randomly selected element
3  k ← PARTITION(A, p, r)  ►Invoke PARTITION method to divide the array
4  return k                ►Return index of pivot in the partitioned array
```

```
RANDOMIZED-QUIKSORT( A, p, r)
1  if  p < r
2    then q← RANDOMIZED-PARTITION(A, p, r)
3         RANDOMIZED-QUIKSORT( A, p, q-1)
4         RANDOMIZED-QUIKSORT( A, q+1, r)
```

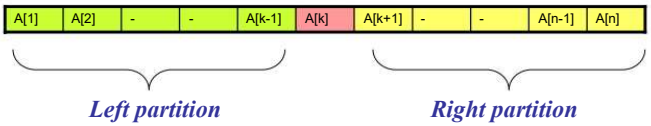
21

Analysis of Quick Sort

22

Quick Sort Analysis

Best Case Scenario



In best case the array is partitioned in to ***two nearly equal subarrays***

The recurrence relation for ***best running time*** is

$T(n) = 2\,T(n/2) + n - 1$

Time to sort array of size n

Time to sort two subarrays each of size n/2

Cost of partitioning subarray of size n-1

23

Quick Sort Analysis

Best case Running Time

The best case running time recurrence

$T(n) = 2\,T(n/2) + n - 1$

can be solved by ***iteration-substitution*** *method*

After k^{th} *iteration:*

$T(n) = 2^k\,T(n/2^k) + k.n - (2^0 + 2^1 + 2^2 + \ldots\ldots + 2^{(k-1)})$

Summing the series

$T(n) = 2^k\,T(n/2^k) + k.n - (2^k - 1)$

Setting $n/2^k = 1$, *gives* $k = \lg n$

$T(n) = 2^{\lg n}.\,T(1) + n.\lg n - (2^{\lg n} - 1)$

$= n + n \lg n - (n - 1)$

$= n \lg n + 1 = \theta(n \lg n)$

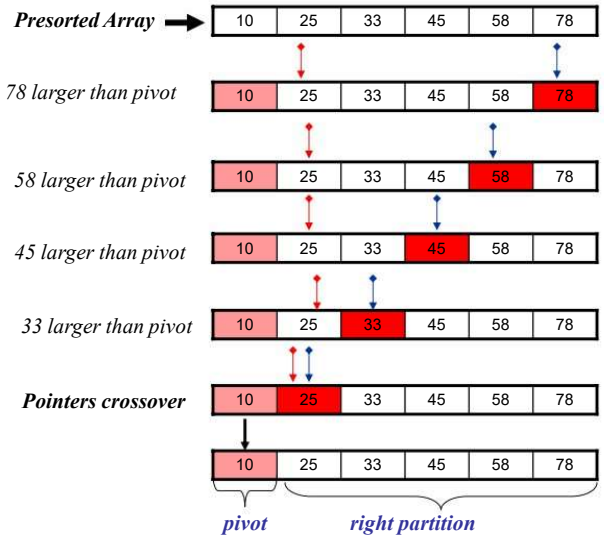
The best case running time of quick sort is $\theta(n \lg n)$

24

Quick Sort Analysis

Worst Case Scenario

The ***worst case*** arises when the input is ***presorted***. In this case array is scanned from right to left, until the pointers crossover. The partitioning results in a single right partition The ***left partition is empty.***

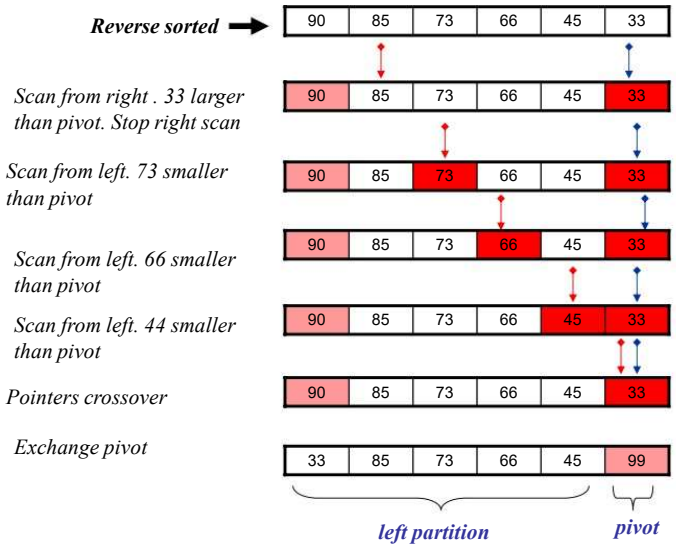


25

Quick Sort Analysis

Worst Case Scenario

The ***worst case*** also arises when input is ***reverse sorted***. In this case the array is scanned from left to right only until pointers crossover. The partitioning results in a single ***left partition***. The ***right partition is empty***

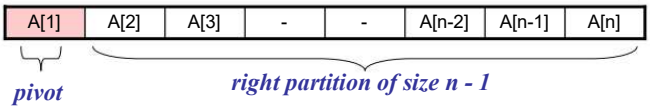


26

Quick Sort Analysis

Worst Case Running Time-I

Consider the case when left partition is empty



The recurrence relation for worst running time is

$T(n) = T(0) + T(n-1) + n - 1$

- $T(n)$: *Time to sort array of size n*
- $T(0)$: *Time to sort left partition T(0)=0*
- $T(n-1)$: *Time to sort right partition of size n-1*
- $n - 1$: *Cost of partitioning subarray of size n-1*

Since ***left partition is empty***, recurrence simplifies to:

$$T(n) = T(n-1) + n - 1, \quad n>1$$

The same recurrence applies when ***right partition is empty***.

27

Quick Sort Analysis

Worst Case Running Time-II

The recurrence

$$T(n) = T(n-1) + n - 1, \quad n > 0$$
$$T(0) = 0$$

can be solved by ***iteration method***

Iteration yields the solution:

$$T(n) = n-1 + (n-2) + (n-3) + \ldots\ldots + 2 + 1$$

$$= n(n-1)/2 = O(n^2)$$

The worst case running time of quick sort is $\theta(n^2)$

28

Avg Time

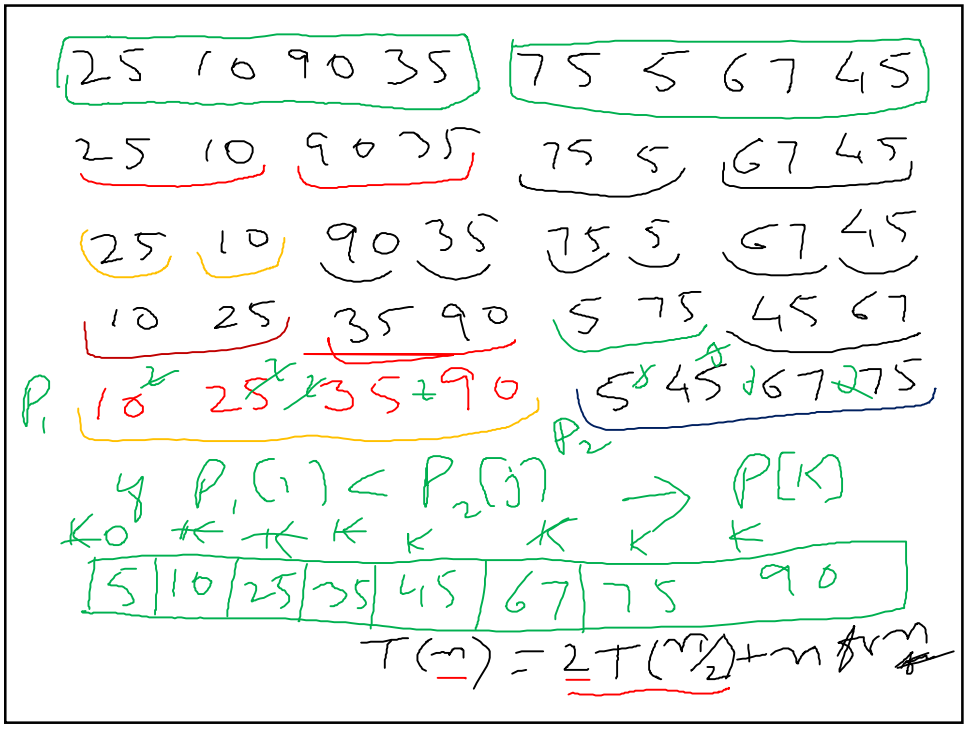
$\Theta(1.39\, n \lg n)$

$= \Theta(n \lg n)$

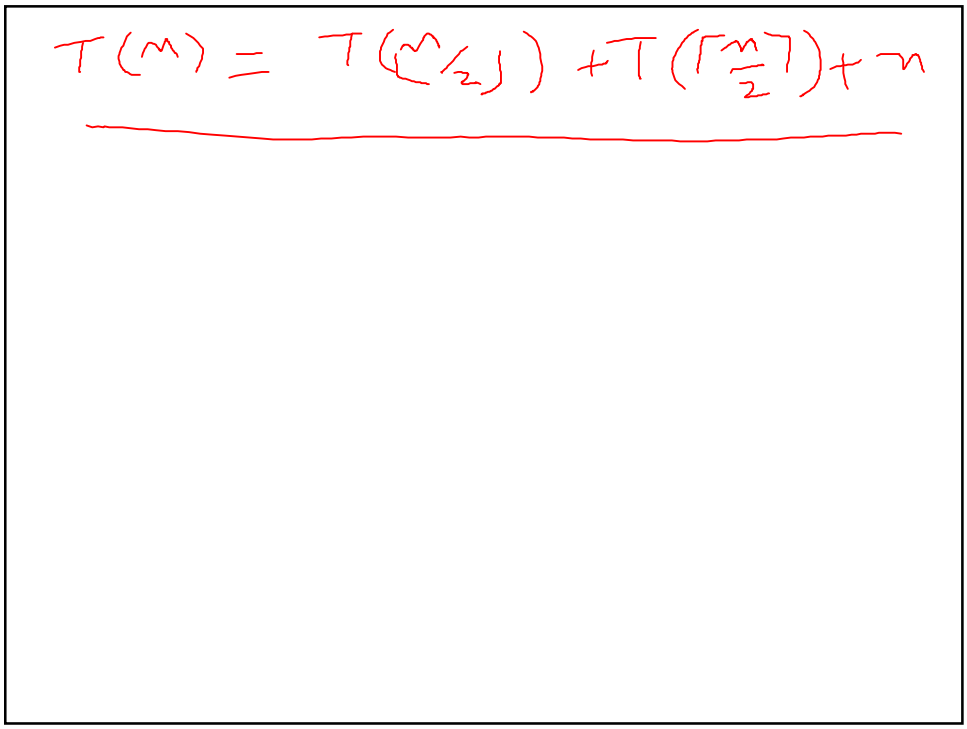
29

Merge Sort

30


25 10 90 35
75 5 67 45
25 10
90 35
75 5
67 45
25
10
90
35
75
5
67
45
10 25
35 90
5 75
45 67
P1
10 25 35 90
P2
5 45 67 75
if P1[i] < P2[j]
P[k]
5 10 25 35 45 67 75 90
T(n) = 2T(n/2) + n

31


T(n) = T(⌊n/2⌋) + T(⌈n/2⌉) + n

32

Merge Sort

Algorithm

The merge sort algorithm is based on divide-and-conquer method.

*The array to be sorted is **partitioned into two sub-arrays of nearly equal size**. The sub-arrays are **sorted and merged***

*In order to sort and merge two **auxiliary arrays**, L and R, are used The left and right partitions of the array A are copied into L and R*

*In order to merge L and R back into A, the elements of L and R are sequentially examined. The **smaller of the two elements is copied back into the array A***

*The procedure is repeated, until **all elements of L or R are exhausted***

*Any **remaining elements** in L or R are simply copied into A*

*The merge sort procedure is applied **recursively.** Thus, array is partitioned into down to sub-arrays, each consisting of a single element*

33

Merge Sort

Pseudo Code

The merge sort algorithm is implemented using two procedures named SORT, MERGE. The SORT procedure partitions the array into two nearly equal sub-arrays, and calls the MERGE procedure. .

```
SORT(A, p, r)
1  if  p < r                    ►Terminate  recursion when left and right partitions overlap
2     then q ← ( p + r ) / 2    ► Divide array into two nearly equal sub-arrays
3          SORT (A, p, q)       ►Recursively sort left partition
4          SORT (A, q+1, r)     ►Recursively sort right partition
5          MERGE (A, p, q, r)   ►Merge sub-arrays
```

34

Merge Sort

Pseudo Code (merging)

The MERGE procedure uses two auxiliary arrays for merging.

```
MERGE(A, p, q, r)
1   n1 ← q - p + 1          ► set n1 to number of elements in the left partition
2   n2 ← r - q              ► set n2 to number of elements in the right partition
3   for i ← 1 to n1
4     do L[i] ← A[p + i - 1]      ► Copy left partition into L
5   for j ← 1 to n2
6     do R[j] ← A[q + j]          ► Copy right partition into R
7   L[n1 + 1] ← ∞           ► Insert a large value into the last cell to mark the end of array L
8   R[n2 + 1] ← ∞           ► Insert a large value into the last cell to mark the end of array R
9   i ← 1                   ► Counter to scan array L , initially set to 1
10  j ← 1                   ► Counter to scan array R, initially set to 1
11  for k ← p to r          ► Process elements from the beginning of left partition to end of right partition
12    do if L[i] ≤ R[j]     ► Identify smaller of the two elements in L and R
13      then A[k] ← L[i]    ► If smaller element in L, copy it into A
14           i ← i + 1      ► Increase counter for L
15      else A[k] ← R[j]    ► If smaller element in R, copy it into A
16           j ← j + 1      ► Increase counter for R
```

35

Analysis of Merge Sort

36

Merge Sort Analysis

Worst Case Running Time

In ***worst case*** the array partitioning and merging is done right up to single element arrays. The running time $T(n)$ for the worst case is given by the recurrence:

$$T(1) = c \quad (c \text{ cost of merging single element})$$

$$T(n) = 2\,T(n/2) + c.n$$

- $T(n)$: *Time to sort array of size n*
- $2\,T(n/2)$: *Time to sort two sub-arrays each of size n/2*
- $c.n$: *Cost of merging two sub-arrays each of size n/2*

The recurrence can be solved by ***substitution method.***

After *kth* iteration

$$T(n) = ck.n + c.2^k T(n/2^k)$$

Setting $n/2^k = 1$, gives $k = \lg n$

Substituting for k gives

$$T(n) = c\,n \lg n + c2^{\lg n} = c.n \lg n + c.\,n$$

The worst case running time is θ***(n lg + n)*** = θ***(nlgn)***

37

Merge Sort Analysis

Average Case Running Time

The average case analysis is complex.

The result of analysis is

$$T(n) = n \lg n - 2n \sum_{i=1}^{\lg n} \frac{1}{2^i + 2} \approx n \lg n - 1.26\,n$$

The average running time of merge sort is θ***(n lg n)***

38